

JavaScript Object-Oriented Programming (OOP) & Classes

RECAP: JAVASCRIPT FUNCTIONS

Before we step into Object-Oriented Programming, let's briefly review JavaScript Functions from our previous module.

A function is a reusable block of code designed to perform a specific task. We define functions to avoid writing the same code repeatedly.

1. Function Declaration

```
function calculateTotal(price, taxRate) {  
    let total = price + (price * taxRate);  
    return total;  
}  
  
// Invoking (calling) the function  
let laptopPrice = calculateTotal(1000, 0.12);  
console.log(laptopPrice); // Output: 1120
```

2. Key Terminology Review

- **Parameters:** Variables listed in the function definition (`price`, `taxRate`).
- **Arguments:** Actual values passed into the function when calling it (`1000`, `0.12`).
- **return Statement:** Stops function execution and sends a value back to the caller.

While functions are great for processing inputs and returning outputs, real-world application data is often complex. Instead of managing separate standalone functions and variables, we can bundle data and behaviors together into **Objects** and **Classes**.

LESSON OVERVIEW

In this chapter, we will learn what JavaScript classes are, how they work, and how to use them to structure scalable programs.

Learning Topics:

1. Object-Oriented Programming (OOP)
2. Classes vs. Objects
3. Class Constructors & Default Parameters
4. Methods & Parameters
5. Private Properties & Encapsulation
6. Getters and Setters (get / set)
7. Inheritance (extends & super)
8. Prototypes (prototype)

1. OBJECT-ORIENTED PROGRAMMING (OOP)



What is OOP?

Object-Oriented Programming (OOP) is a programming paradigm (a way of organizing code) where code is structured into **objects**. An object bundles **properties** (data/variables) and **methods** (actions/functions) together.

Why use OOP?

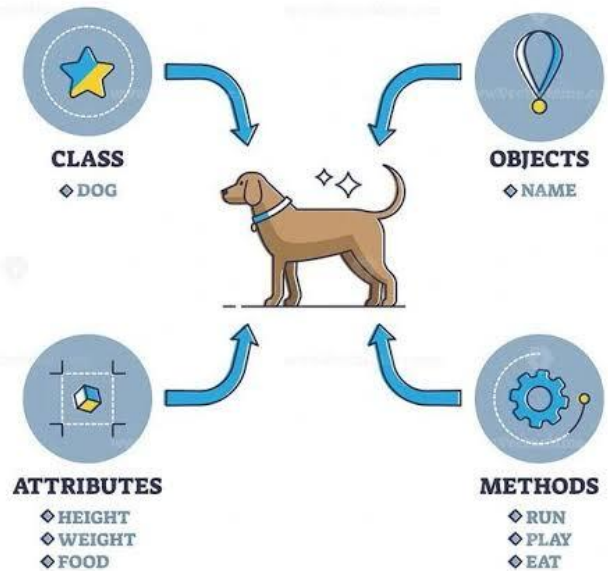
- **Reusability:** Write a blueprint once, create as many instances as you need.
- **Maintainability:** If you need to fix a bug or add a feature across all similar items, you only modify the central template.
- **Complexity Reduction:** Reduces spaghetti code by grouping related functions and variables together.

Real-World Analogy

Imagine designing an application for a pet shelter.

- An **Animal** in general has generic properties like name, weight, height, and colors.
- If you create a **Fish** or a **Dog**, you can reuse all generic Animal properties and add species-specific properties (e.g., breed for dogs, waterType for fish).

OBJECT ORIENTED PROGRAMMING



2. CLASSES AND OBJECTS

Objects vs. Classes

- **Object:** A concrete collection of properties (keys) and values.
- **Class:** A template or blueprint used to build objects.

Standard Object Literal (Refresher)

Here is how we previously declared a single object:

```
1  let dog = {  
2    dogName: "JavaScript",  
3    weight: 2.4,  
4    color: "brown",  
5    breed: "chihuahua"  
6  };
```

Constructor Function (The Old Way)

Beneath the surface, JavaScript classes are actually wrapped inside special constructor functions. Historically, objects were instantiated using constructor functions like this:

```
function Dog(dogName, weight, color, breed) {  
    this.dogName = dogName;  
    this.weight = weight;  
    this.color = color;  
    this.breed = breed;  
}  
  
let dog1 = new Dog("Jacky", 30, "brown", "labrador");
```

Modern Class Syntax (The ES6 Way)

Modern JavaScript introduces the `class` keyword to provide cleaner and more readable syntax:

```
class Dog {  
    constructor(dogName, weight, color, breed) {  
        this.dogName = dogName;  
        this.weight = weight;  
        this.color = color;  
        this.breed = breed;  
    }  
}  
  
let dog2 = new Dog("JavaScript", 2.4, "brown", "chihuahua");  
  
console.log(dog2.dogName, "is a", dog2.breed, "and weighs", dog2.weight, "kg.");
```

OUTPUT:

```
JavaScript is a chihuahua and weighs 2.4 kg.
```

3. CONSTRUCTORS AND DEFAULT VALUES

The **constructor** Method

The **constructor()** is a special built-in method that automatically runs when a **new** instance of a class is created using the new keyword.

- A class can contain **only one constructor** method.
- The **this** keyword refers to the specific instance of the object being created.

```
class Person {  
  constructor(firstname, lastname) {  
    this.firstname = firstname;  
    this.lastname = lastname;  
  }  
}  
  
let p1 = new Person("Maaike", "van Putten");  
console.log("Hi", p1.firstname);
```

Output: Hi Maaike

What happens if parameters are missing?

If you instantiate an object without supplying all arguments, missing values default to **undefined**:

```
let p2 = new Person("Maaike");  
console.log("Hi", p2.firstname, p2.lastname);
```

Output: Hi Maaike undefined

Commercial break, this is for your attendance Asynchronous Class(Thursday), if you use javascript classes what you prefer? Using old way or Modern Class Syntax (The ES6 Way) ? explain your answer send it to your private threads.

Using Default Parameters

To prevent **undefined** outputs, assign default values directly inside the constructor signature:

```
class PersonWithDefault {  
  constructor(firstname, lastname = "Doe") {  
    this.firstname = firstname;  
    this.lastname = lastname;  
  }  
}  
  
let p3 = new PersonWithDefault("Maaike");  
console.log("Hi", p3.firstname, p3.lastname);
```

Output: Hi Maaike Doe

4. CLASS METHODS

In JavaScript Object-Oriented Programming (OOP), a **class** acts like a blueprint for creating objects.

Methods are simply functions that belong to a class and define what actions an object created from that class can perform.

I. Syntax Rule: No **function** Keyword

Inside a class body, you do not write the word function when defining a method. You simply write the name of the method followed by parentheses () and curly braces {}.

Incorrect: `function greet(){ ... }`

Correct: `greet(){ ... }`

```
class Person {
  constructor(firstname, lastname) {
    this.firstname = firstname;
    this.lastname = lastname;
  }

  // Instance method without external parameters
  greet() {
    console.log("Hi there! I'm", this.firstname);
  }

  // Instance method with parameters and a return value
  compliment(name, object) {
    return "That's a wonderful " + object + ", " + name;
  }
}

let p = new Person("Maaïke", "van Putten");

// Calling the greet method
p.greet();
// Output: Hi there! I'm Maaïke

// Calling the compliment method and storing its output
let complimentMsg = p.compliment("Harry", "hat");
console.log(complimentMsg);
```

II. Line-by-Line Code Breakdown

The Constructor:

```
constructor(firstname, lastname) {
  this.firstname = firstname;
  this.lastname = lastname;
}
```

- The **constructor** is a special method that runs automatically when a new object is created.
- **this.firstname** saves the provided **firstname** value directly into the object's memory.

Method 1:

`greet()` (No Parameters)

```
greet() {  
  console.log("Hi there! I'm", this.firstname);  
}
```

- **Purpose:** Allows the object to print a personalized greeting to the console.
- **Key Concept:** `this.firstname` accesses the `firstname` property saved inside the current object instance.

Method 2:

`compliment(name, object)` (With Parameters & Return Value)

```
compliment(name, object) {  
  return "That's a wonderful " + object + ", " + name;  
}
```

- **Purpose:** Takes external inputs (`name` and `object`) and returns a combined text string.
- **Key Concept:** Unlike `greet()`, this method sends a value back using the `return` keyword instead of printing directly to the console.

III. Creating an Instance and Calling Methods

```
let p = new Person("Maaike", "van Putten");  
p.greet();
```

1. **Instantiation:** `new Person("Maaike", "van Putten")` creates a new object stored in variable `p`. Its `firstname` property is set to `"Maaike"`.
2. **Method Execution:** `p.greet()` calls the `greet()` method on object `p`.
3. **Output:** The console outputs: `Hi there! I'm Maaike`

Quick Summary Table

Component	Code	Description
Object Creation	<code>let p = new Person(...)</code>	Creates an individual instance from the Person class blueprint.
Method Call	<code>p.greet()</code>	Executes the greet action for the specific object <code>p</code> .
this Keyword	<code>this.firstname</code>	Points to the properties of the specific object running the code.

5. PRIVATE PROPERTIES (#) AND ENCAPSULATION

Encapsulation is the Object-Oriented Programming (OOP) principle of hiding an object's internal data from direct outside access. This protects sensitive information and prevents external code from messing up internal values.

1. Public vs. Private Properties

- **Public (Default):** Accessible anywhere inside or outside the class.
- **Private (# symbol):** Accessible **only** inside the class where they are declared.

2. Code Breakdown: Declaring & Protecting Private Data

```
class Person {  
  // 1. Declare private fields at the top of the class  
  #firstname;  
  #lastname;  
  
  constructor(firstname, lastname) {  
    // 2. Assign values using the # prefix  
    this.#firstname = firstname;  
    this.#lastname = lastname;  
  }  
}  
  
let p = new Person("Maria", "Saga");  
console.log(p.firstname); // Output: undefined
```

- **Step 1:** You must explicitly list #firstname; and #lastname; at the top of the class body before using them.
- **Step 2:** Inside the constructor, prefix the field names with # (this.#firstname = firstname).
- **Why undefined?** Trying to read p.firstname fails because the property is actually #firstname. JavaScript blocks outside access to private fields to protect them. (Note: Trying p.#firstname outside the class will trigger a syntax error.)

3. Code Breakdown: Constructor Validation & Getter Methods

```
class ValidatedPerson {  
  #firstname;  
  #lastname;  
  
  constructor(firstname, lastname) {  
    if (firstname.startsWith("M")) {  
      this.#firstname = firstname;  
    } else {  
      this.#firstname = "M" + firstname; // Forces the name to start with 'M'  
    }  
    this.#lastname = lastname;  
  }  
  
  getFirstName() {  
    return this.#firstname;  
  }  
}  
  
let person1 = new ValidatedPerson("kay", "Moon");  
console.log(person1.getFirstName()); // Output: Mkay
```

- **Validation in Action:** The constructor checks if firstname starts with "M". Since "kay" does not start with "M", the else block runs, prefixing "M" to make it "Mkay".
- **Getter Method (getFirstName):** Because #firstname is private, external code cannot view it directly. The class provides getFirstName() to controlledly expose that value.

Key Concepts Comparison

Feature	Code Syntax	Access Point	Purpose
Private Field	<code>#fieldname;</code>	Inside the class body only	Prevents external tampering
Getter Method	<code>getFirstName() { return this.#firstname; }</code>	Call via instance (person1.getFirstName())	Safely exposes private data
Validation	<code>if (condition) { ... } inside constructor</code>	Triggered automatically on new creation	Ensures data fits required form

6. GETTERS AND SETTERS

Getters and setters are special keywords (**get** and **set**) that allow developer-created methods to be accessed as if they were regular object properties. They allow control over how private fields (**#firstname**) are read or modified, without changing how external code accesses them.

1. Core Concepts

- **get (Getter):** Used to read and return a private property value. It takes **zero parameters**.
- **set (Setter):** Used to update or modify a private property value. It takes **exactly one parameter**.

2. Line-by-Line Code Breakdown

```
class Person {
  #firstname;
  #lastname;

  constructor(firstname, lastname) {
    this.#firstname = firstname;
    this.#lastname = lastname;
  }

  // GETTER: Reads #firstname
  get firstname() {
    return this.#firstname;
  }

  // SETTER: Updates #firstname with custom logic
  set firstname(firstname) {
    if (firstname.startsWith("M")) {
      this.#firstname = firstname;
    } else {
      this.#firstname = "M" + firstname;
    }
  }
}
```

- **How the Getter Works:** When `p.firstname` is accessed, JavaScript automatically invokes `get firstname()` behind the scenes to safely return `this.#firstname`.
- **How the Setter Works:** When assigning `p.firstname = "Adnane"`, JavaScript passes `"Adnane"` into `set firstname(firstname)`. Because `"Adnane"` does not start with `"M"`, the `else` block runs, appending `"M"` to make it `"MAdnane"`.

3. Execution & Crucial Syntax Rules

```
let p = new Person("Maria", "Saga");

// Reading data (Getter)
console.log(p.firstname); // Output: Maria

// Updating data (Setter)
p.firstname = "Adnane";
console.log(p.firstname); // Output: MAdnane
```

- **Use Property Syntax, Not Method Syntax:** Even though `get` and `set` are written like methods inside the class, they are invoked like standard variables (`p.firstname`), not functions.
- **Common Student Pitfall:** Calling a getter with parentheses (e.g., `p.firstname()`) will throw a `TypeError: p.firstname is not a function`.

Summary Comparison

Feature	Regular Method	Getter (get)	Setter (set)
Declaration Syntax	<code>getFirstName()</code>	<code>get firstname()</code>	<code>set firstname(value)</code>
Invocation Syntax	<code>p.getFirstName()</code>	<code>p.firstname</code>	<code>p.firstname = "Value"</code>
Parameters	0 or more	Strictly 0	Strictly 1
Primary Purpose	Performs an action	Reads private data	Validates/writes private data

Next lesson INHERITANCE and PROTOTYPES